Diagnosing Memory and Performance Problems

9 min read Updated 28 Jul 2026 Topic 47 of 65

Bookmark Mark done

Why it matters

This is the topic that most clearly separates someone who has run Java in production from someone who has only written it. "The service is slow and memory keeps climbing — what do you do?" has a methodical answer, and being able to give it in order, naming the tools, is worth more in a senior interview than any amount of syntax knowledge.

The concept, simply

Every JVM investigation follows the same shape: **observe, capture, analyse, verify**. The mistake is jumping to a fix — raising `-Xmx`, adding threads — before knowing which of the handful of possible causes you actually have.

The tools ship with the JDK, which is worth saying because people reach for third-party agents unnecessarily:

Tool	Answers
`jcmd`	Everything — the modern front end for all of it
`jstat -gcutil`	Is GC the problem? Live generation occupancy
Thread dump	What are threads doing? Deadlock? Contention?
Heap dump + Eclipse MAT	What is retaining memory, and who holds it?
JFR + JDK Mission Control	Low-overhead continuous profiling — the best default

Native Memory Tracking	Heap looks fine but the process is huge — where?

How it works

The decision tree for "memory keeps growing":

1. **Which OOM message?** Heap space, Metaspace, native thread and GC overhead have four different causes (previous topic).
2. **Is the heap actually full?** `jstat -gcutil`. Old generation near 100% after a full GC means genuinely live data.
3. **Is it growing monotonically?** A leak grows and never recovers; a busy application sawtooths and returns to a stable baseline.
4. **Heap dump and analyse.** Eclipse MAT's dominator tree names the retainer; path-to-GC-roots names who holds it.
5. **If the heap is fine but the process is huge**, it is off-heap — Native Memory Tracking, direct buffers, or a native library.

For "the service is slow", the equivalent first question is whether threads are *blocked* or *busy*. A thread dump answers it in seconds: many threads `BLOCKED` on one monitor is lock contention; threads `RUNNABLE` in application code is CPU work; threads `TIMED_WAITING` on a socket is a slow dependency.

Code examples

The commands, in the order you would actually run them:

BASH

Copy

```
jcmd -l                                 # find the pid

# 1. Is it GC?  S0 S1 E O M CCS YGC YGCT FGC FGCT GCT
jstat -gcutil <pid> 1000 10
#    O (old gen) pinned near 100 with FGC climbing = full and not recoveri

# 2. What are the threads doing?
```

```
jcmd <pid> Thread.print > threads.txt
#    a few seconds apart — one is a snapshot, three show
#    whether a thread is stuck or merely busy.

# 3. Capture the heap. ⚠ This PAUSES the JVM for seconds to minutes and
#    writes a file the size of the heap. Do it on one instance, out of rot
jcmd <pid> GC.heap_dump /var/tmp/heap.hprof

# 4. Off-heap suspicion (needs -XX:NativeMemoryTracking=summary at startup
jcmd <pid> VM.native_memory summary

# 5. Best default for ongoing work — JFR, ~1% overhead, safe in production
jcmd <pid> JFR.start name=diag settings=profile duration=120s filename=/tm
#    Open in JDK Mission Control: allocation by call site, lock contention
#    GC pauses, hot methods, I/O — all in one recording.
```

Two leak patterns you will meet, with their fixes:

JAVA Copy

```java
// 1. The unbounded static cache. A static field is a GC ROOT, so nothing
//    in it is ever collected.
private static final Map<String, byte[]> CACHE = new HashMap<>();
// ✓ Use a bounded cache with eviction — Caffeine, or LinkedHashMap with
//   removeEldestEntry (Chapter 4).

// 2. The ThreadLocal on a pooled thread. Pool threads live for the life o
//    the application, so a value never removed is never collected — and i
//    also pins the class loader, which is the classic redeploy leak.
private static final ThreadLocal<Context> CTX = new ThreadLocal<>();
public void handle(Request r) {
    CTX.set(new Context(r));
    try { process(); }
    finally { CTX.remove(); }        // ✓ non-negotiable on a pooled threc
}
```

What a thread dump tells you at a glance:

TEXT Copy

```
// Lock contention — 40 threads in this state, all on the same monitor:
"http-nio-8080-exec-7" java.lang.Thread.State: BLOCKED (on object monitor)
    at com.app.Cache.get(Cache.java:42)
    - waiting to lock <0x76b2f1234> (a com.app.Cache)

// Slow dependency — threads parked in socket reads, not blocked on locks:
"http-nio-8080-exec-9" java.lang.Thread.State: RUNNABLE
    at java.net.SocketInputStream.socketRead0(Native Method)

// Deadlock — the JVM detects and names it explicitly:
Found one Java-level deadlock: …
```

Under the hood

JFR is the tool worth adopting if you take one thing from this topic. Java Flight Recorder is built into the JVM, costs roughly 1% overhead, and records allocation by call site, lock contention, GC events, exceptions, I/O and method profiles into a single file. It has been free and open-source since Java 11. Running it continuously in production means that when something goes wrong you already have the data, instead of trying to reproduce the problem while it is happening.

Heap dumps are expensive and safe. They pause the JVM and write a file as large as the live heap — on a 16 GB heap that is minutes of downtime and 16 GB of disk. Take one from an instance removed from the load balancer, and analyse it elsewhere. What makes them worth it is that a heap dump contains the answer: MAT's dominator tree ranks objects by *retained* size, which is the memory that would be freed if that object went away, so the leak is usually the first or second row.

Beware the observer effect. Attaching a sampling profiler with a naive agent can distort exactly the timings you are measuring, and running a debugger disables the JIT optimisations that cause many concurrency bugs. Low-overhead tools built on safepoint-free sampling — JFR, async-profiler — give a truer picture.

The methodical answer to give in an interview: reproduce or observe with metrics first, form a hypothesis about which region or resource is exhausted, capture the specific

than that the load happened to drop.

Common mistakes

- **Raising `-Xmx` as a first response.** It buys time and hides the cause; unbounded growth will exhaust any heap.
- **Restarting without capturing anything.** The evidence is destroyed and the problem returns.
- **Taking one thread dump.** Three, seconds apart, distinguish stuck from busy.
- **Heap-dumping a live production instance.** It pauses the JVM — take the instance out of rotation.
- **Guessing instead of measuring.** Intuition about performance is reliably wrong.
- **Not running with GC logs and `HeapDumpOnOutOfMemoryError`.** Free, and you will want both.
- **Forgetting `ThreadLocal.remove()` on pooled threads.** Leaks the value and pins the loader.
- **Skipping verification.** Confirm the fix with the same measurement that showed the problem.

Interview lens

> **Q: "Memory keeps growing in production. Walk me through it."**
> A: First, which OOM or which metric — heap, Metaspace and native thread failures have different causes. Then `jstat -gcutil` to see whether the old generation stays near full after a full GC, and whether the trend is monotonic or a healthy sawtooth. If it is genuinely full, a heap dump from an instance out of rotation, analysed in Eclipse MAT: the dominator tree ranks by retained size and path-to-GC-roots names the holder. If the heap looks fine but the process is large, it is off-heap, and I use Native Memory Tracking.

A: A thread dump, three of them a few seconds apart. That immediately distinguishes the three common cases: many threads `BLOCKED` on one monitor is lock contention, threads `RUNNABLE` in application code is CPU-bound work, and threads waiting in socket reads is a slow downstream dependency. It costs nothing and narrows the search enormously before I reach for a profiler.

Q: "What tools do you use?"

A: Mostly what ships with the JDK. `jcmd` as the front end, `jstat` for live GC statistics, thread and heap dumps for the two dump-based investigations, Eclipse MAT for heap analysis, and Native Memory Tracking for off-heap. My default in production is JFR with JDK Mission Control — about 1% overhead and it records allocation by call site, lock contention, GC and hot methods, so the data already exists when something goes wrong.

Q: "What are the most common causes of a Java memory leak?"

A: Unbounded static collections, since a static field is a GC root; `ThreadLocal`s never removed on pooled threads, which also pin the class loader and cause Metaspace leaks on redeploy; listeners and callbacks registered and never unregistered; and caches with no eviction policy. All of them are "still reachable, no longer useful", which is exactly what a garbage collector cannot help with.

Q: "Is taking a heap dump safe in production?"

A: It works, but it pauses the JVM for the duration and writes a file the size of the live heap — minutes and many gigabytes on a large one. So I take it from an instance pulled out of the load balancer, or configure `-XX:+HeapDumpOnOutOfMemoryError` so one is captured automatically at the moment of failure, which is usually better evidence anyway.

A: By re-running the same measurement that identified the problem, under comparable load, and confirming the metric changed — old-generation occupancy after full GC, pause times, the retained size of the suspect object. Skipping that step is how teams conclude a fix worked when traffic simply dropped.

Cheat-card

- Method: **observe → capture → analyse → verify**. Never raise `-Xmx` first.
- Memory: which OOM → `jstat -gcutil` → monotonic or sawtooth? → **heap dump + MAT**.
- Slow: **three thread dumps**. BLOCKED = contention · RUNNABLE = CPU · socket read = dependency.
- MAT: **dominator tree** (retained size) + **path to GC roots** (who holds it).
- Heap fine, process huge → **Native Memory Tracking**.
- **JFR + Mission Control** — ~1% overhead, run it continuously.
- Leaks: static collections · `ThreadLocal` **without** `remove()` · unregistered listeners · unbounded caches.
- Heap dumps **pause the JVM** — take the instance out of rotation.
- Always ship with GC logs + `-XX:+HeapDumpOnOutOfMemoryError`.

QUICK CHECK — no marks, just for you.

1. The service has become slow. What is the cheapest first diagnostic?

- ◯ A heap dump
- ◯ Restarting with a larger heap
- ◯ Enabling a sampling profiler in production

Three thread dumps a few seconds apart — they distinguish lock contention, CPU-bound work and a slow dependency immediately

Check answer

2. Which is a leading cause of Java memory leaks?

- A ThreadLocal never removed on a pooled thread — the pool thread lives for the application's life, so the value is never collected and it pins the class loader
- Creating too many short-lived objects
- Not calling System.gc() often enough
- Circular references between objects

Check answer

3. Is taking a heap dump safe on a live production instance?

- Yes, it has no measurable effect
- It pauses the JVM for the duration and writes a file the size of the live heap — take it from an instance out of rotation, or configure -XX:+HeapDumpOnOutOfMemoryError
- No, it always crashes the JVM
- Only if the heap is under 1 GB

Check answer

LEARN MORE

ARTICLES & DOCS

- Troubleshooting guide — Java SE 17
  docs.oracle.com
- jcmd — man page
  docs.oracle.com
- JDK Mission Control
  www.oracle.com

VIDEOS



Best Java GC Algorithm: G1 vs ZGC vs Parallel (How to Choose) - Webinar
yCrash

Hand-picked references for this lesson.